國立臺北科技大學
National Taipei University of Technology

電子工程系碩士班

碩士學位論文

Department of Electronic Engineering

Master Thesis

非合作式 PCI 設備模擬

Non-cooperative PCI Device Emulation

研究生：蘇翊端

Researcher: Yi-Tuan Su

指導教授：賴冠廷 博士

Advisor: Kuan-Ting Lai, Ph.D.

中華民國一百一十五年七月

「學位論文口試委員會審定書」掃描檔

審定書填寫方式以系所規定為準，但檢附在電子論文內的掃描檔須具備以下條件：

1. 含指導教授、口試委員及系所主管的完整簽名。
2. 口試委員人數正確，碩士口試委員至少 3 人、博士口試委員至少 5 人。
3. 若此頁有論文題目，題目應和書背、封面、書名頁、摘要頁的題目相符。
4. 此頁有無浮水印皆可。

審定書不用頁碼

摘要

關鍵詞：虛擬化、設備模擬、VirtIO、PCI transport

輕量虛擬機器透過最小化映像檔來降低資源佔用與攻擊面，但同時移除了除錯與監控等工具。vmsh 針對此問題提出非合作式（non-cooperative）的解決方案，能在不修改虛擬機器監視器（hypervisor）與客體作業系統（Guest OS）的前提下，將 VirtIO 設備動態掛載至運行中的虛擬機器，為其補充所需的服務與工具。然而，vmsh 與 Cloud Hypervisor 在中斷機制上存在不相容，限制了 vmsh 的 hypervisor 支援範圍。

本研究擴充 vmsh 的功能，在維持非合作式與 hypervisor 無關（hypervisor-agnostic）的特性前提下，為 vmsh 實現了 VirtIO over PCI transport，並利用 KVM 所提供的 KVM_SIGNAL_MSI 介面實現 MSI-X 中斷的注入，來解決 vmsh 在中斷機制上與 Cloud Hypervisor 的不相容問題。在 PCI 設備模擬方面，本研究延伸 vmsh 的 side-load library 機制在客體作業系統內部觸發 PCI rescan，並透過 ptrace 攔截客體作業系統發出的 Port I/O 指令，使其得以發現由 vmsh 所模擬的 PCI 虛擬設備，進而透過標準 virtio-pci 驅動程式與該設備進行 I/O 操作。

此外，本研究在 QEMU 與 Cloud Hypervisor 兩種環境下完成功能驗證，並使用 xfstests 確認本研究中 PCI transport 實現的初步正確性，使 vmsh 成功支援 Cloud Hypervisor，擴展了 vmsh 的 hypervisor 支援範圍，並驗證了使用非合作式方法模擬 PCI 設備的可行性。

ABSTRACT

Keywords: Virtualization, Device Emulation, VirtIO, PCI Transport

Lightweight virtual machines reduce resource consumption and attack surface by minimizing their images, but at the cost of removing debugging and monitoring tools. vmsh addresses this problem with a non-cooperative approach that dynamically attaches VirtIO devices to running virtual machines without modifying the hypervisor or the guest OS, supplementing VMs with the required services and tools. However, vmsh is incompatible with Cloud Hypervisor due to differences in interrupt mechanisms, limiting the range of hypervisors that vmsh can support.

This study extends vmsh's functionality by implementing VirtIO over PCI transport while preserving its non-cooperative and hypervisor-agnostic properties. This study leverages the KVM_SIGNAL_MSI system call interface provided by KVM to inject MSI-X interrupts, resolving the interrupt mechanism incompatibility between vmsh and Cloud Hypervisor. For PCI device emulation, this study extends vmsh's side-load library mechanism to trigger PCI rescan within the guest OS, and intercepts Port I/O instructions issued by the guest via ptrace, enabling the guest OS to discover the PCI virtual device emulated by vmsh and perform I/O operations through the standard virtio-pci driver.

Furthermore, this study conducts functional verification under both QEMU and Cloud Hypervisor environments, and uses xfstests to confirm the preliminary correctness of the PCI transport implementation. As a result, vmsh successfully supports Cloud Hypervisor, expanding its hypervisor compatibility and demonstrating the feasibility of emulating PCI devices using a non-cooperative approach.

TAIPEI TECH

誌謝

所有對於研究提供協助之人或機構，作者都可在誌謝中表達感謝之意。

TAIPEI TECH

目錄

摘要 i

ABSTRACT ii

誌謝 iii

目錄 iv

圖目錄 v

表目錄 vi

第一章　緒論（大標） 1

1.1　研究動機與背景（小標） 1

1.1.1　研究背景 (小小標) 1

第二章　若標題太長，則可以分成兩行排列的形式撰寫 5

2.1　名詞定義（小標） 5

2.2　模型說明（小標） 5

2.3　VMSH: Hypervisor-agnostic Guest Overlays for VMs 6

2.3.1　System Architecture 6

第三章　Experiment and Result 8

3.1　Functional Validation and Generality 8

3.1.1　PCI Device Discovery and Initialization 8

3.1.2　Block I/O and Filesystem Validation 8

3.1.3　Cloud Hypervisor Validation 8

參考文獻 11

TAIPEI TECH

圖目錄

1.1 Cool train station 2

1.2 “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” 2

1.3 Cool train station 3

1.4 Cool train station 4

2.1 Cool train station 5

2.2 System architecture of vmsh. vmsh establishes devices in the guest through side-loading a kernel library. The block device supports the overlay root file system, and the console device handles the I/O of the spawned process. Numbers 1–4 correspond to the four-step workflow. (Adapted from [2]) 6

3.1 dmesg log of PCI device discovery and BAR assignment (QEMU) 8

3.2 dmesg log of VirtIO negotiation completion (QEMU) 8

3.3 Guest userspace query of the vdb device 8

3.4 Block I/O read/write and data integrity test (QEMU) 9

3.5 ext4 filesystem mount test (QEMU) 9

3.6 Data integrity test (Cloud Hypervisor) 9

3.7 ext4 filesystem mount test (Cloud Hypervisor) 10

TAIPEI TECH

表目錄

1.1 表格範例標題 1
1.2 表格範例標題 3

2.1 表格範例標題 5
2.2 Host machine specifications 7

TAIPEI TECH

第一章　緒論（大標）

1.1　研究動機與背景（小標）

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{1.1}$$

1.1.1　研究背景 (小小標)

背景內文背景內文背景內文背景內文, 背景內文背景內文背景內文背景內文背景內文背景內文，如表 1.1 所示。

表 1.1: 表格範例標題

Protocol	P	CS_1	CS_2	RG
SD	$O(1), O(1)$, N/A	$O(n-t), O(1)$, N/A	$O(n-t), O(1)$, N/A	$O(1), O(n), O(n)$
MSSMul	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$
MSSAdd	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	N/A, N/A, N/A	$O(1), O(n), O(n)$
SC	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$

1.1.1.1　研究動機 (小小標)

$$(1+x)^n = 1 + \frac{nx}{1!} + \frac{n(n-1)x^2}{2!} \tag{1.2}$$

動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機。

動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機。動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機。動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機動機，如圖 1.1、圖 1.2 所示。

動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機。

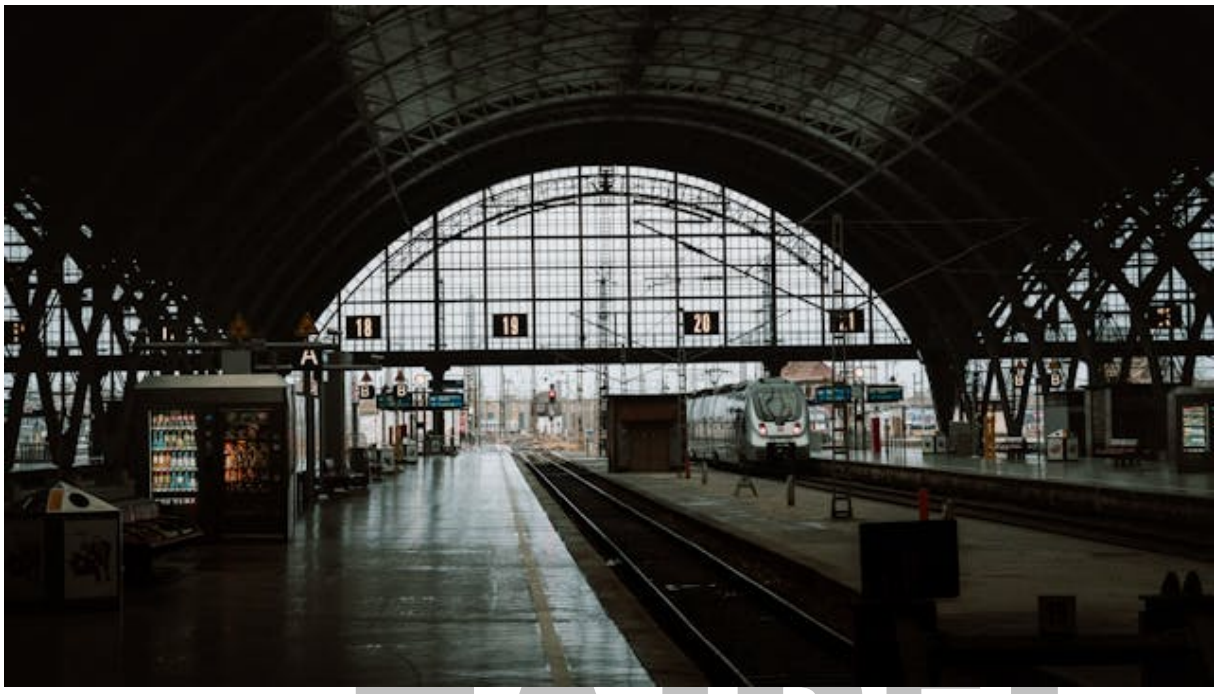

圖 1.1: Cool train station

圖 1.2: “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station” “Cool train station”

圖 1.3: Cool train station

表 1.2: 表格範例標題

Protocol	P	CS_1	CS_2	RG
MSSMul	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$
MSSAdd	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	N/A, N/A, N/A	$O(1), O(n), O(n)$
SC	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$
MSSMul	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$
MSSAdd	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	N/A, N/A, N/A	$O(1), O(n), O(n)$
SC	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$

動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機。動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機。

動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機。動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機。動機動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機。

動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機。動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機。動機動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機。動機動機動機動機動機，動機動機動機動機動機動機動機動機動機動機動機動機動機動機，動機動機動機動機動機動機動機動機動機。

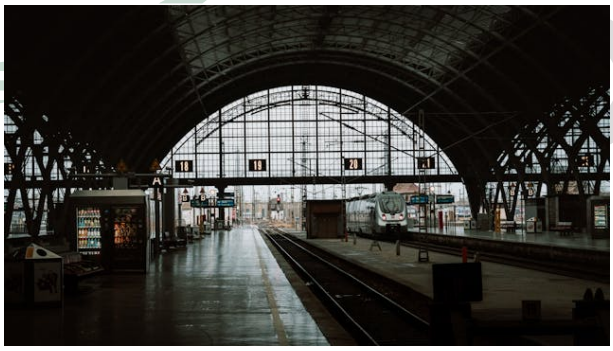

圖 1.4: Cool train station

第二章　若標題太長，則可以分成兩行排列的形式撰寫

2.1　名詞定義（小標）

定義定義定義定義定義定義 [1]，定義定義定義定義，定義定義定義定義定義定義定義定義定義定義，定義定義。

表 2.1: 表格範例標題

Protocol	P	CS_1	CS_2	RG
MSSMul	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$
SC	$O(1), O(1)$, N/A	$O(n-t), O(n), O(1)$	$O(n-t), O(n)$, N/A	$O(1), O(n), O(n)$

2.2　模型說明（小標）

說明說明說明說明，說明說明說明說明說明說明說明說明說明說明說明說明說明說明，說明說明說明說明說明說明說明說明說明。

圖 2.1: Cool train station

2.3 VMSH: Hypervisor-agnostic Guest Overlays for VMs

2.3.1 System Architecture

As an independent host process, vmsh runs alongside the hypervisor on the same host machine. vmsh does not modify the hypervisor's source code, nor does it rely on any management API provided by the hypervisor. Instead, vmsh completes all operations externally through ptrace and the underlying KVM API. Figure 2.2 presents the system architecture of vmsh (adapted from [2] Figure 2), whose workflow can be divided into the following four steps:

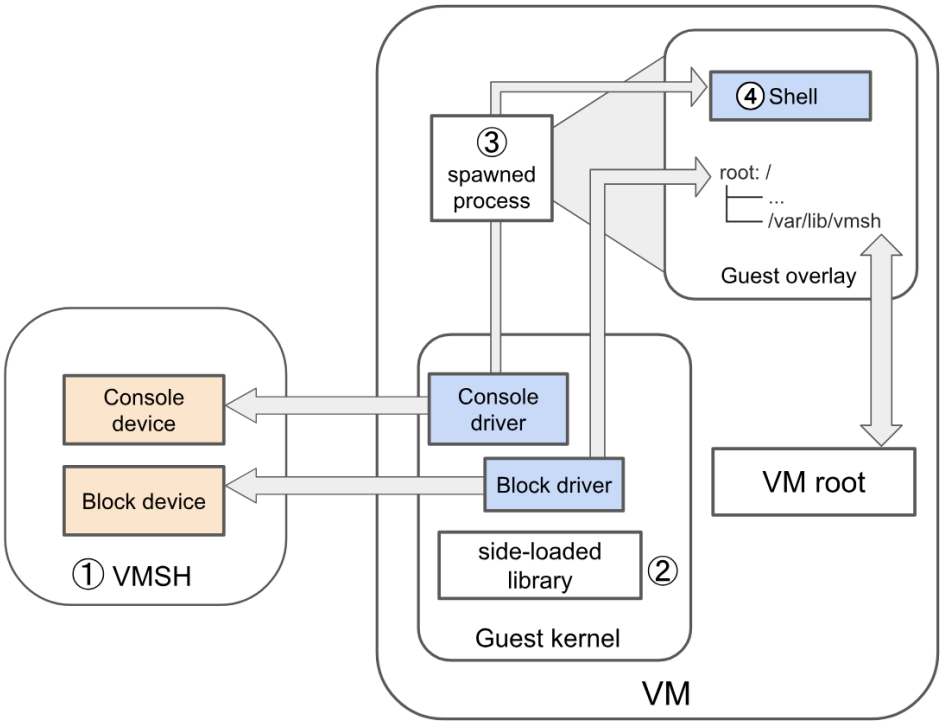

圖 2.2: System architecture of vmsh. vmsh establishes devices in the guest through side-loading a kernel library. The block device supports the overlay root file system, and the console device handles the I/O of the spawned process. Numbers 1–4 correspond to the four-step workflow. (Adapted from [2])

All experiments in this study were conducted on the same physical host machine, whose

hardware and operating system specifications are listed in Table 2.2.

表 2.2: Host machine specifications

Item	Specification
CPU	Intel Core i7-11700 @ 2.50GHz (8 cores, 16 threads)
Memory	48 GB DDR4
Storage	ADATA SX8200PNP 480GB NVMe SSD
OS	Ubuntu 22.04 LTS (kernel 5.15.0)

TAIPEI TECH

第三章　Experiment and Result

3.1　Functional Validation and Generality

3.1.1　PCI Device Discovery and Initialization

```
[root@qemu-microvm:~]# dmesg | tail -25
[   47.007513] stage1: detected linux version 6.3.6
[   47.007514] stage1: using PCI transport (EXPERIMENTAL)
[   47.007514] stage1: [EXPERIMENTAL] triggering PCI bus rescan
[   47.009020] pci 0000:00:0a.0: [1af4:1042] type 00 class 0x010000
[   47.009710] pci 0000:00:0a.0: reg 0x10: [mem 0x00000000-0x00000fff]
[   47.010314] pci 0000:00:0a.0: reg 0x18: [mem 0x00000000-0x00000fff]
[   47.010921] pci 0000:00:0a.0: reg 0x20: [mem 0x00000000-0x00000fff]
[   47.011512] pci 0000:00:0a.0: reg 0x30: [mem 0x00000000-0x000007ff pref]
[   47.021659] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021663] pci 0000:00:0a.0: BAR 0: assigned [mem 0x20000000-0x20000fff]
[   47.021777] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021779] pci 0000:00:0a.0: BAR 2: assigned [mem 0x20001000-0x20001fff]
[   47.021897] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021898] pci 0000:00:0a.0: BAR 4: assigned [mem 0x20002000-0x20002fff]
[   47.022007] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.022008] pci 0000:00:0a.0: BAR 6: assigned [mem 0x20003000-0x200037ff pref]
[   47.022869] virtio-pci 0000:00:0a.0: enabling device (0000 -> 0002)
```

圖 3.1: dmesg log of PCI device discovery and BAR assignment (QEMU)

```
[   47.052456] virtio_blk virtio2: [vdb] 131072 512-byte logical blocks (67.1 MB/64.0 MiB)
```

圖 3.2: dmesg log of VirtIO negotiation completion (QEMU)

```
[root@qemu-microvm:~]# ls /dev/vd*
/dev/vda  /dev/vdb

[root@qemu-microvm:~]# cat /sys/block/vdb/size
131072
```

圖 3.3: Guest userspace query of the vdb device

3.1.2　Block I/O and Filesystem Validation

3.1.3　Cloud Hypervisor Validation

```
[root@qemu-microvm:~]# dd if=/dev/vdb of=/dev/null bs=4k count=1
1+0 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 0.00445993 s, 918 kB/s

[root@qemu-microvm:~]# dd if=/dev/zero of=/dev/vdb bs=4k count=1
1+0 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 0.00578631 s, 708 kB/s

[root@qemu-microvm:~]# echo "VMSH_TEST_DATA" | dd of=/dev/vdb bs=512 count=1
0+1 records in
0+1 records out
15 bytes copied, 0.0124918 s, 1.2 kB/s

[root@qemu-microvm:~]# dd if=/dev/vdb bs=512 count=1 | head -c 20
1+0 records in
1+0 records out
VMSH_TEST_DATA
512 bytes copied, 0.00840111 s, 60.9 kB/s
```

圖 3.4: Block I/O read/write and data integrity test (QEMU)

```
[root@qemu-microvm:~]# lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
fd0   2:0    1     4K  0 disk
sr0  11:0    1  1024M  0 rom
vda 254:0    0 267.5M  1 disk /nix/store
                               /
vdb 254:16   0    10M  0 disk

[root@qemu-microvm:~]# mkdir -p /tmp/mnt

[root@qemu-microvm:~]# mount /dev/vdb /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
bin  dev  etc  lost+found  nix  proc  sys  tmp  var

[root@qemu-microvm:~]# umount /tmp/mnt
```

圖 3.5: ext4 filesystem mount test (QEMU)

```
[root@qemu-microvm:~]# echo "VMSH_CH_TEST_2026" | dd of=/dev/vdb bs=512 count=1
0+1 records in
0+1 records out
18 bytes copied, 0.00511157 s, 3.5 kB/s

[root@qemu-microvm:~]# dd if=/dev/vdb bs=512 count=1 | head -c 20
1+0 records in
1+0 records out
VMSH_CH_TEST_2026
512 bytes copied, 0.00541082 s, 94.6 kB/s
```

圖 3.6: Data integrity test (Cloud Hypervisor)

```
[root@qemu-microvm:~]# lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
vda 254:0    0 267.5M  1 disk /nix/store
                              /
vdb 254:16   0    10M  0 disk

[root@qemu-microvm:~]# mkdir -p /tmp/mnt

[root@qemu-microvm:~]# mount /dev/vdb /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
bin  dev  etc  lost+found  nix  proc  sys  tmp  var

[root@qemu-microvm:~]# umount /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
```

圖 3.7: ext4 filesystem mount test (Cloud Hypervisor)

參考文獻

[1] Leslie Lamport. *LaTeX: a Document Preparation System*. 2nd ed. Massachusetts: Addison Wesley, 1994.

[2] Jörg Thalheim et al. “vmsh: Hypervisor-agnostic Guest Overlays for VMs”. In: *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys ’22)*. Rennes, France: ACM, 2022, pp. 678–693.

TAIPEI TECH